



GitLab CI/CD: A Practical Guide for Test, Build, and Deploy

🕒 Date de création	@16 janvier 2024 14:27
👤 Créée par	 Alexon
▼ Type	Information

- [GitLab CI/CD](#)
- [GitLab CI/CD job to Run Tests](#)
- [Configure Container Registry in Scaleway](#)
- [Test : Push Docker image From your local computer Terminal window](#)
- [Push Docker image from GitLab CI/CD](#)
 - [Step 1: Obtain Registry Information](#)
 - [Step 2: Configure GitLab CI/CD to Use the External Registry](#)
- [Deployment on Remote Servers with SSH and Docker-Compose](#)
 - [Step 1 : Generate SSH keys](#)
 - [Step 2 : Import SSH keys to Scaleway](#)
 - [Step 3 : Set CI/CD variables in Gitlab CI/CD](#)
 - [Step 4 : CI/CD Jobs to Connect to machine via SSH and perform deployments](#)
 - [Step 5: Add Staging and Production Environment](#)
- [Sources](#)

GitLab CI/CD

A Practical Guide for Automating Testing, Building and Deployment using GitLab CI/CD.

GitLab CI/CD job to Run Tests

We use pytest, a testing framework to do unit test, functional tests before building and deployment of containers. We generate also coverage report and test report to display directly in the gitlab.

Example of job to run tests

```
test:
  stage: test
  extends:
    - .environment-django-ci
  script:
    # # Run pytest with generation both HTML and Cobertura XML
    # if no tests found, convert exit code 5 to 0
    - pytest --cov=. --cov-report html --cov-report xml:coverage.xml
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml
      junit: report.xml
  paths:
    - htmlcov/
    - coverage.xml
    - report.xml
```

Configure Container Registry in Scaleway

Create a namespace that will contain all our images

Container Registry

Namespaces 1

Vos quotas Registry Besoin d'une mise à jour ? Contactez notre support.

ALL ▾

>

Namespaces: 1 / 99999

Utilisation des données: 0 o / 300 To

Estimation des frais: 0,00 €

● Stockage

● Transfert

Les namespaces sont une collection d'images de conteneur, chacune portant l'identifiant unique du namespace donné.
[Documentation Quickstart Container Registry](#) [🔗](#)

+ Créer un namespace

☐	Nom	Politiques de confidentialité	Modifié	Images	Taille	Région	
☒	dap-projects	🔒	il y a 11 minutes	0	0 o	🇫🇷 PAR	...

It will ask you to create an API key in the API Keys tab of your Identity and Access Management (IAM) dashboard. use the Secret Key as your password.

Test : Push Docker image From your local computer Terminal window

```
docker login rg.fr-par.scw.cloud/dap-projects -u nologin --password-stdin
docker pull ubuntu:latest
docker tag ubuntu:latest rg.fr-par.scw.cloud/dap-projects/ubuntu:latest
docker push rg.fr-par.scw.cloud/dap-projects/ubuntu:latest
```

Push Docker image from GitLab CI/CD

Step 1: Obtain Registry Information

First, you need the following details from your external container registry:

- **Registry URL:** The address of the container registry.
- **Credentials:** Username and password or API token for accessing the registry.

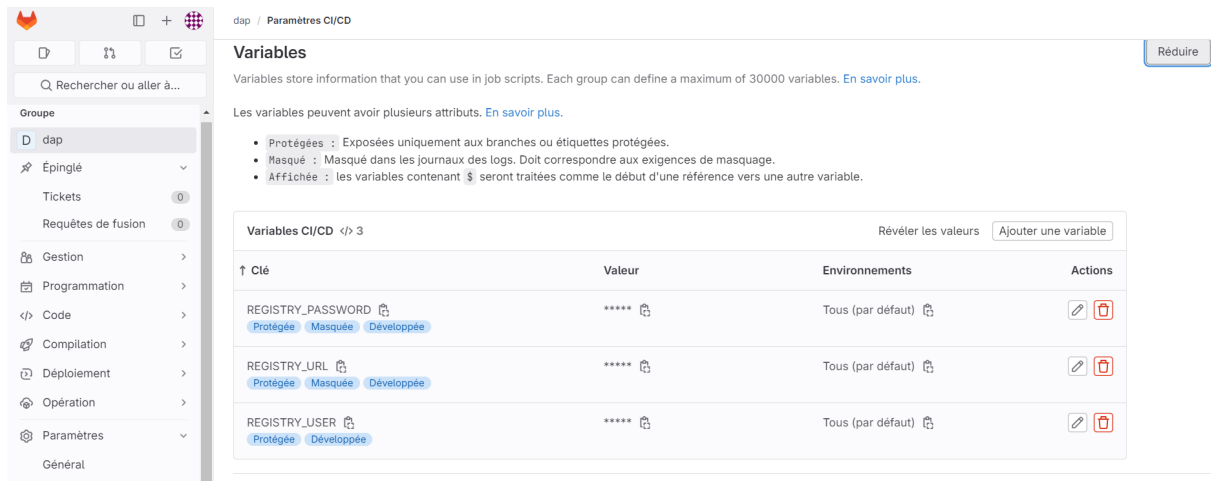
Step 2: Configure GitLab CI/CD to Use the External Registry

In your GitLab repository, you'll

configure the `.gitlab-ci.yml` file to authenticate and interact with the external container registry:

1. Add Registry Credentials to CI/CD Variables:

- Go to your project in GitLab or Go to your Group (if you want to add Group level Variables)
- Navigate to **Settings > CI / CD**.
- Expand the **Variables** section.
- Add your external registry credentials here. For example, add variables for `REGISTRY_URL`, `REGISTRY_USER`, and `REGISTRY_PASSWORD`.
- Ensure that the "Mask variable" option is selected for sensitive data like passwords or tokens to keep them hidden in job logs.



group level Variables

Example :

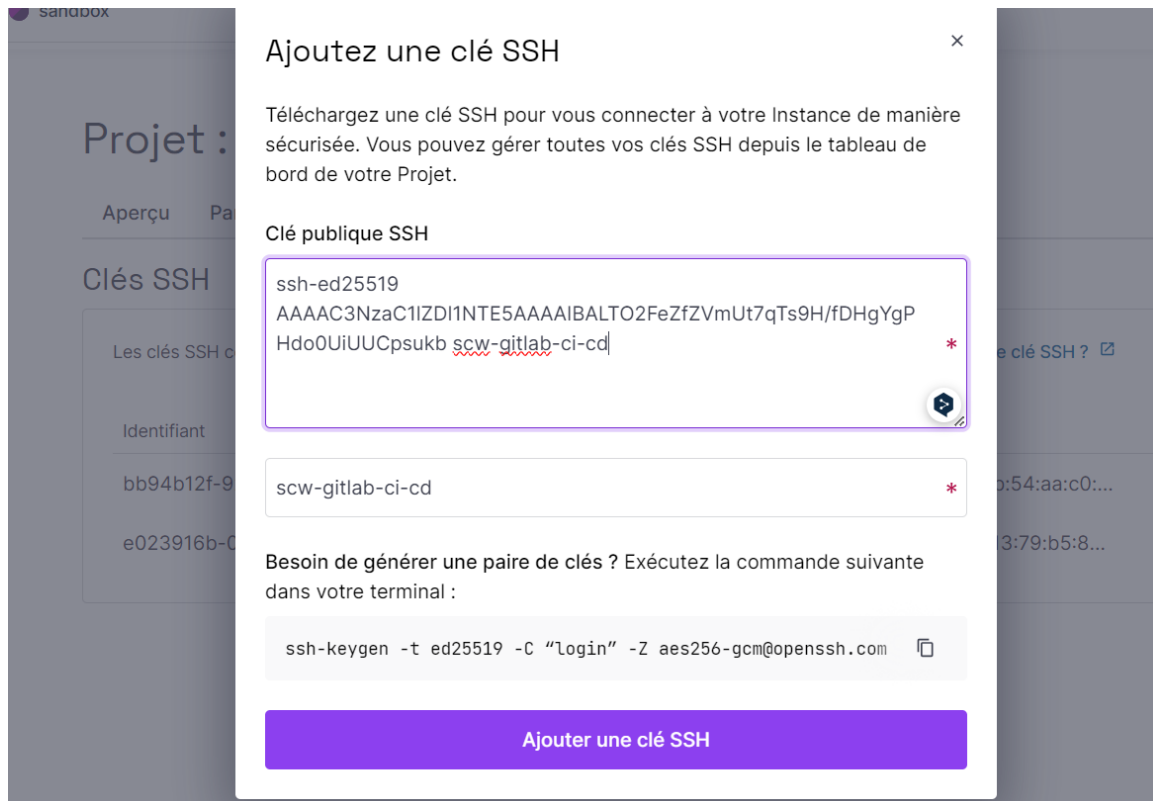
```
build:
  image: docker:latest
  services:
    - docker:dind
  script:
    - echo $REGISTRY_PASSWORD | docker login $REGISTRY_URL -u
    - docker build -t $REGISTRY_URL/my-image:$CI_COMMIT_REF_NAME
    - docker push $REGISTRY_URL/my-image:$CI_COMMIT_REF_NAME
```

Deployment on Remote Servers with SSH and Docker-Compose

Step 1 : Generate SSH keys

```
ssh-keygen -t ed25519 -C "scw-gitlab-ci-cd" -f ~/.ssh/scw-gitlab-ci-cd
# the passphrase is needed in the CI/CD job
```

Step 2 : Import SSH keys to Scaleway



Step 3 : Set CI/CD variables in Gitlab CI/CD

- Add SSH connection related variables.
 - SSH key
 - SSH Host and port : IP/Domain of testing/staging/production server, port
- ENV variables for the app (e.g, `ENV_TESTING` variable contains all the environment variable required for the app for testing environment)

put this content value as `ENV_TESTING` key and check box the protected variables.

```
SECRET_KEY="django-insecure-=y=)0jm5+51$^gt7reakr73e*jnet0
```

```
# Database
DB_NAME=boilerplate
DB_USER=postgres
DB_PASSWORD=postgres
DB_HOST=db
DB_PORT=5432
```

```
.....
```

```
SSH_APP_PATH : /opt/app/back
```

```
.....
```

- In the group variables, make sure to add Container Registry related variables.

Les variables peuvent avoir plusieurs attributs. [En savoir plus.](#)

- **Protégées** : Exposées uniquement aux branches ou étiquettes protégées.
- **Masqué** : Masqué dans les journaux des logs. Doit correspondre aux exigences de masquage.
- **Affichée** : les variables contenant `$` seront traitées comme le début d'une référence vers une autre variable.

Variables CI/CD </> 12				Révéler les valeurs	Ajouter une variable
↑ Clé	Valeur	Environnements	Actions		
ENV_CI  Protégée	***** 	Tous (par défaut) 	 		
ENV_DEV  Protégée	***** 	Tous (par défaut) 	 		
ENV_LOCAL  Protégée	***** 	Tous (par défaut) 	 		
ENV_PROD  Protégée	***** 	Tous (par défaut) 	 		
ENV_STAGING  Protégée	***** 	Tous (par défaut) 	 		
ENV_TESTING  Protégée	***** 	Tous (par défaut) 	 		
SSH_APP_PATH  Protégée Développée	***** 	Tous (par défaut) 	 		
SSH_KEY_PASSPHRASE  Protégée Masquée Développée	***** 	Tous (par défaut) 	 		
SSH_PRIVATE_KEY  Protégée Développée	***** 	Tous (par défaut) 	 		
SSH_TESTING_HOST  Protégée Développée	***** 	Tous (par défaut) 	 		
SSH_TESTING_PORT  Protégée Développée	***** 	Tous (par défaut) 	 		
SSH_USER  Protégée Développée	***** 	Tous (par défaut) 	 		

Variables du groupe (héritées)

Ces variables sont héritées du groupe parent.

Variables CI/CD </> 3			
Clé	Attributs	Environnements	Groupe
REGISTRY_PASSWORD  Protégée Masquée Développée		Tous (par défaut) 	dap
REGISTRY_USER  Protégée Développée		Tous (par défaut) 	dap
REGISTRY_URL  Protégée Masquée Développée		Tous (par défaut) 	dap

Step 4 : CI/CD Jobs to Connect to machine via SSH and perform deployments

Adapt your docker compose files based on the variables like image with specific name and tag

```
api:
  image: $REGISTRY_URL/$API_IMAGE_NAME:${API_IMAGE_TAG:-latest}
```

you have to put these variables in a .env because docker compose by default will load .env file.

Here is an example of a deployment job, which includes several steps to ensure correct deployment

```
deploy-testing:
  stage: deploy
  image: ubuntu:latest
  environment:
    name: testing
    url: https://api.testing.boilerplate.appftth.fr
  variables:
    API_IMAGE_NAME: boilerplate/api
    API_IMAGE_TAG: $CI_COMMIT_SHORT_SHA
    WEB_API_IMAGE_NAME: boilerplate/webapi
    WEB_API_IMAGE_TAG: latest
    SSH_CMD: "ssh -p $SSH_TESTING_PORT $SSH_USER@$SSH_TESTING_HOST"
    SCP_CMD: "scp -P $SSH_TESTING_PORT"
  script:
    - # Setup SSH
    - which ssh-agent || ( apt update -y && apt install openssh-client )
    - eval $(ssh-agent -s)
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
    - echo 'echo "$SSH_KEY_PASSPHRASE"' > ~/.ssh/.print_ssh_password
    - chmod 700 ~/.ssh/.print_ssh_password
    - chmod +x ~/.ssh/.print_ssh_password
    - eval $(ssh-agent -s)
    - echo "$SSH_PRIVATE_KEY" | tr -d '\r' | DISPLAY=:0.0 ssh-keygen -f ~/.ssh/id_rsa -N ""
    - >
    - ssh-keyscan -p $SSH_TESTING_PORT $SSH_TESTING_HOST >> ~/.ssh/known_hosts
    - || (echo "Error: Could not reach $SSH_TESTING_HOST:$SSH_TESTING_PORT")
```

```

- chmod 644 ~/.ssh/known_hosts
- $SSH_CMD "echo 'Deploying to server'"

# Preparing the server
- $SSH_CMD "docker system prune -a -f || true"
- $SSH_CMD "cd $SSH_APP_PATH && docker compose -f docker-
- $SSH_CMD "rm -rf /tmp/app || true && mkdir -p /tmp/app
- $SSH_CMD "rm -rf $SSH_APP_PATH && mkdir -p $SSH_APP_PATH

# Copy necessary files
- $SCP_CMD $CI_PROJECT_DIR/docker-compose.testing.yml $SSH
- echo "$ENV_TESTING" > .env
- $SCP_CMD .env $SSH_USER@$SSH_TESTING_HOST:$SSH_APP_PATH

# Set up environment variables on the server
# put REGISTRY_URL, images, tags in the .env file
- $SSH_CMD "echo \"REGISTRY_URL=$REGISTRY_URL\" >> $SSH_A
- $SSH_CMD "echo \"API_IMAGE_NAME=$API_IMAGE_NAME\" >> $S
- $SSH_CMD "echo \"API_IMAGE_TAG=$API_IMAGE_TAG\" >> $SSH
- $SSH_CMD "echo \"WEB_API_IMAGE_NAME=$WEB_API_IMAGE_NAME
- $SSH_CMD "echo \"WEB_API_IMAGE_TAG=$WEB_API_IMAGE_TAG\"

# Docker login and start containers
- $SSH_CMD "echo \"$REGISTRY_PASSWORD\" | docker login $R
- $SSH_CMD "cd $SSH_APP_PATH && docker compose -f docker-

# Health check
- $SSH_CMD "curl -s http://localhost:8000/healthcheck | g
only:
- main

```

Step 5: Add Staging and Production Environment

- Make the above CI/CD job to make it reusable to Staging and Production deployment.

- make sure to add **Wildcard protected branch on** `release-v*` like that `release-vX.X.X` can access CI/CD protected variables.
 - **settings > Repository > Expand Protected branches > Add protected branch > Type** `release-v*` **> Put at least Maintainers for permissions > Click on Protect**

add also, `feature/*`, `bugfix/*`, `hotfix/*` branches, if you plan to run CI/CD jobs on them.

- create a release branch like `release-v1.0.0` and push to GitLab
- After reviewing the app meaning after the validation of staging environment, you would deploy the app on the production environment. you can deploy it By playing the `deploy-production`.

État	Pipeline	Créé par	Étapes
✓ Réussi 00:03:25 il y a un jour	fix jobs #101 main c5825d2e plus récent		✓ ✓ ✓ ↓
⛔ Bloqué 00:03:42 il y a un jour	fix jobs #99 release-v2.0.0 c5825d2e plus récent		✓ ✓ ⚙ ▶ ↓ ↓ deploy-production

- By clicking in Environments You could also see all the environments here , in **environments (e.g, testing, staging, production)**

B boilerplate_back

🔗

🔔

☆ Ajouter aux favoris 0

🍴 Fork 0

⋮

🔗 112 validations

🌿 2 branches

🏷 0 étiquette

💾 12,1 Mo stockage de projet

🌐 3 environnements

🔗 allow failure to false

Alexon Uthayakumar rédigé il y a 10 minutes

✓ 53298fd8

main boilerplate_back / +

Historique

Rechercher un fichier

Modifier

Code

LISEZMOI

Configuration de CI / CD

Ajouter une LICENCE

Ajouter un CHANGELOG

Ajouter un CONTRIBUTING

Auto DevOps activé

Ajouter une grappe de serveurs Kubernetes

Ajouter une page wiki

Configurer les intégrations

click on environments

GitLab CI/CD: A Practical Guide for Test, Build, and Deploy

11

Actif 3

Arrêté 0

Nettoyer les environnements

Activer les applications de revue

Nouvel environnement

Rechercher par nom d'environnement

>

production

Ouvrir

Arrêter

>

staging

Ouvrir

Arrêter

>

testing

Ouvrir

Arrêter

we have 3 environments : testing, staging and production

Final Step

- Make sure the merge requests can be merged only if pipelines succeeded

Settings → Merge requests → Merge checks → Pipelines must succeed

Merge checks

These checks must pass before merge requests can be merged.

- ☒ Pipelines must succeed

Merge requests can't be merged if the latest pipeline did not succeed or is still running.

 - ☐ Skipped pipelines are considered successful

Introduces the risk of merging changes that do not pass the pipeline.
 - ☐ All threads must be resolved

Sources

GitLab CI/CD jobs for Testing

Test coverage visualization | GitLab

GitLab product documentation.

 https://docs.gitlab.com/16.7/ee/ci/testing/test_coverage_visualization.html

Unit test reports | GitLab

GitLab product documentation.

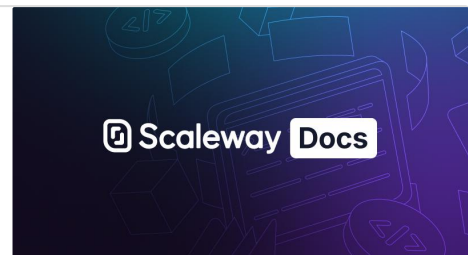
 https://docs.gitlab.com/16.7/ee/ci/testing/unit_test_reports.html

Scaleway Container Registry

Container Registry - Quickstart

This page shows you how to get started with Scaleway Container Registry.

 <https://www.scaleway.com/en/docs/containers/container-registry/quickstart/>



GitLab CI/CD jobs for Deployment

Predefined CI/CD variables reference | GitLab

GitLab product documentation.

 https://docs.gitlab.com/16.7/ee/ci/variables/predefined_variables.html

Using SSH keys with GitLab CI/CD | GitLab

GitLab product documentation.

 https://docs.gitlab.com/ee/ci/ssh_keys/

Environments and deployments | GitLab

GitLab product documentation.

 <https://docs.gitlab.com/ee/ci/environments/>